■ Apache Spark

Top 50 Toughest Scenario-Based

Interview Questions

Asked at Netflix · Uber · LinkedIn · Databricks · Meta · Google · Amazon

50	7	FAANG+	Expert
Questions	Categories	Companies	Level

■ Table of Contents

Section 1	Performance Optimization & Tuning	Q1–Q8
Section 2	Memory Management & Spill	Q9–Q14
Section 3	Shuffles, Joins & Skew Handling	Q15–Q22
Section 4	Streaming & Fault Tolerance	Q23–Q30
Section 5	Delta Lake, Lakehouse & Storage	Q31–Q37
Section 6	Cluster Management & Scheduling	Q38–Q43
Section 7	Advanced APIs, SQL & Architecture	Q44–Q50

Section 1: Performance Optimization & Tuning

Q1 — Your Spark job reads 10 TB of Parquet data daily but takes 4 hours. Profiling shows 80% of time is in a single wide transformation. How do you diagnose and fix this?

■ *Asked at: Netflix · Databricks · Uber* | ■ *Difficulty: Hard*

■ Answer:

Start by checking the Spark UI for the slow stage — look at shuffle read/write bytes, spill metrics, and task duration variance. 80% in one stage usually means data skew or an expensive shuffle. Steps: (1) Check if partition count is adequate — spark.sql.shuffle.partitions default 200 is too low for 10 TB; set to 2000-4000. (2) Use .explain() to verify the physical plan — look for BroadcastHashJoin vs SortMergeJoin. (3) Enable AQE (spark.sql.adaptive.enabled=true) to auto-coalesce small partitions and handle skew. (4) If Parquet columns are selectively read, verify predicate pushdown is happening. (5) Check for UDFs that block Catalyst optimization — replace with built-in functions.

■ Key Code / Config:

```
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true')
spark.conf.set('spark.sql.shuffle.partitions', '3000')
# Analyze plan
df.explain(mode='cost')
```

■ *Always check spill-to-disk metrics first — they indicate memory pressure, not just slow logic.*

Q2 — You have a DataFrame with 500 million rows. After a groupBy().agg(), output has only 200 partitions but downstream operations are slow. What is happening and how do you fix it?

■ *Asked at: LinkedIn · Google · Amazon* | ■ *Difficulty: Hard*

■ Answer:

The groupBy().agg() shuffle produces partitions sized by spark.sql.shuffle.partitions (default 200). With 500M rows, each partition may hold millions of records causing OOM or slow tasks. Fix: (1) Increase shuffle partitions proportionally to data size. (2) With AQE enabled, use spark.sql.adaptive.coalescePartitions.enabled=true to auto-merge small ones. (3) After aggregation, use repartition(n) or coalesce(n) strategically. (4) If downstream is a write, use partitionBy() on a low-cardinality column to align physical layout with query patterns.

■ Key Code / Config:

```
spark.conf.set('spark.sql.shuffle.partitions', '2000')
result = df.groupBy('user_id').agg(F.sum('amount'))
# Repartition for downstream efficiency
result.repartition(500).write.parquet('output/')
```

■ *Rule of thumb: target 100–200 MB per partition. Calculate: total_shuffle_bytes / target_partition_size.*

Q3 A pipeline uses .cache() extensively but memory usage keeps growing until executors OOM. Describe what is wrong and the correct caching strategy.

■ *Asked at: Meta · Airbnb · Twitter* | ■ *Difficulty: Hard*

■ Answer:

Common mistakes: (1) Caching DataFrames derived from wide transformations without unpersisting — each .cache() pins that lineage. (2) Using MEMORY_AND_DISK when data exceeds executor memory causes excessive GC. (3) Not calling .unpersist() after the cached DF is no longer needed. Correct strategy: (a) Only cache DFs reused 2+ times. (b) Choose storage level based on reuse frequency and size — MEMORY_ONLY for hot small DFs, MEMORY_AND_DISK_SER for large ones. (c) Always unpersist explicitly. (d) Monitor via spark.catalog.clearCache() in tests. (e) Prefer checkpointing for iterative ML loops over caching.

■ Key Code / Config:

```
df_cached = df.persist(StorageLevel.MEMORY_AND_DISK_SER)
# ... multiple uses ...
df_cached.unpersist()  # Always clean up

# For iterative algorithms, prefer checkpoint
sc.setCheckpointDir('hdfs://checkpoints/')
df.checkpoint()
```

■ *Use Spark UI Storage tab to see fraction cached and memory consumed per RDD.*

Q4 Your Spark SQL query on Delta Lake is slow despite Z-ordering. Explain when Z-ordering helps and when it doesn't, and what alternatives exist.

■ *Asked at: Databricks · Snowflake · Palantir* | ■ *Difficulty: Medium*

■ Answer:

Z-ordering co-locates related data in files to enable data skipping via min/max statistics. It helps when: (1) Queries filter on the Z-ordered columns with high selectivity. (2) Data isn't already clustered on those columns. It doesn't help when: (1) Cardinality is too low (e.g., boolean columns). (2) Filters use non-Z-ordered columns. (3) Table is small enough to scan fully. Alternatives: (a) Liquid Clustering (Delta 3.1+) — auto-manages clustering without full rewrites. (b) Partition pruning for high-cardinality date/region columns. (c) Bloom filters for exact-match lookups on high-cardinality string columns.

■ Key Code / Config:

```
-- Z-order on query-critical columns
OPTIMIZE events ZORDER BY (user_id, event_date);

-- Bloom filter for string lookups
ALTER TABLE events SET TBLPROPERTIES (
  'delta.dataSkippingNumIndexedCols' = '32',
  'delta.bloomFilter.columns' = 'session_id'
);
```

■ *Z-ordering is most effective when combined with partition pruning on a coarser column (e.g., partition by date, Z-order by user_id).*

Q5 You notice that a Spark job's executor GC time is >30% of total task time. What are the root causes and remediation steps?

■ *Asked at: Uber · DoorDash · Lyft* | ■ *Difficulty: Hard*

■ Answer:

High GC indicates memory pressure — too many objects are being created and promoted to Old Gen. Root causes: (1) Large number of small Java objects (e.g., Row objects from RDD API). (2) Insufficient executor memory relative to data being processed. (3) Using Kryo serialization is not enabled. (4) String interning or accumulator misuse creating object retention. Remediation: (a) Switch to Dataset/DataFrame API — Tungsten's off-heap columnar format reduces GC pressure. (b) Enable Kryo: spark.serializer=org.apache.spark.serializer.KryoSerializer. (c) Increase spark.executor.memory or decrease spark.memory.fraction to give more heap to user memory. (d) Use off-heap memory: spark.memory.offHeap.enabled=true. (e) Tune GC: use G1GC with -XX:+UseG1GC.

■ Key Code / Config:

```
spark.conf.set('spark.serializer', 'org.apache.spark.serializer.KryoSerializer')
spark.conf.set('spark.memory.offHeap.enabled', 'true')
spark.conf.set('spark.memory.offHeap.size', '4g')
# JVM options in spark-defaults.conf:
# spark.executor.extraJavaOptions -XX:+UseG1GC -XX:G1HeapRegionSize=16m
```

■ *Check Spark UI Executor tab for GC time column. Target <5% GC time.*

Q6 How do you tune Spark for a job that reads 1 TB of JSON, transforms it, and writes Parquet? Walk through the complete configuration strategy.

■ *Asked at: Amazon · Google · Microsoft* | ■ *Difficulty: Hard*

■ Answer:

Step-by-step: (1) Parallelism on read: JSON doesn't split well — use spark.sql.files.maxPartitionBytes=128MB and ensure enough cores. Prefer pre-converting to Parquet/Avro. (2) Set shuffle partitions: approx 1000-2000 for 1 TB. (3) Enable AQE for dynamic optimization. (4) Broadcast small lookup tables (<10 MB). (5) Use columnar Parquet output with snappy/zstd compression. (6) Executor sizing: 4-8 cores per executor, 16-32 GB memory, not too many small executors. (7) Enable speculative execution for straggler tasks. (8) For AWS EMR/cloud: use instance storage for shuffle (not EBS) via spark.local.dir.

■ Key Code / Config:

```
spark.conf.set('spark.sql.files.maxPartitionBytes', str(128*1024*1024))
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.shuffle.partitions', '1500')
spark.conf.set('spark.speculation', 'true')

df.write.option('compression', 'zstd') \
        .partitionBy('date') \
        .parquet('s3://bucket/output/')
```

■ *zstd compression gives better ratio than snappy with similar decompression speed — prefer it for cold storage.*

Q7 What is the Tungsten execution engine and how does it differ from the legacy RDD execution model? When does it not help?

■ *Asked at: Netflix · Spotify · Stripe* | ■ *Difficulty: Medium*

■ Answer:

Tungsten is Spark's physical execution layer that bypasses JVM object overhead using: (1) Off-heap binary memory management — stores data in compact binary format, eliminating Java object header overhead (~16 bytes/object). (2) Cache-aware computation — sort/hash algorithms designed for CPU cache locality. (3) Whole-stage code generation (WSCG) — compiles multiple operators into a single JVM function, eliminating virtual function calls. Differences from RDD: RDD uses Java serialization and per-record function calls; Tungsten uses vectorized columnar processing. Tungsten doesn't help when: (a) Using Python UDFs — data must serialize to/from Python. (b) Using RDD API directly. (c) Complex Hive UDFs that bypass Catalyst.

■ Key Code / Config:

```
# Tungsten WSCG can be checked via explain
df.explain(mode='codegen')

# Disable to debug (not for production)
spark.conf.set('spark.sql.codegen.wholeStage', 'false')
```

■ *Replace Python UDFs with Pandas UDFs (vectorized) to keep data in Arrow format and minimize Python serialization overhead.*

Q8 Explain Adaptive Query Execution (AQE). Describe three real-world scenarios where AQE significantly improves performance.

■ *Asked at: Databricks · Cloudera · Hortonworks* | ■ *Difficulty: Hard*

■ Answer:

AQE re-optimizes the query plan at runtime using actual statistics collected after each shuffle stage, rather than relying on estimated stats. Three key features: (1) Dynamic partition coalescing — after a shuffle, if actual partition sizes are small, AQE merges them to avoid too many tiny tasks. Scenario: after aggressive filtering, 2000 shuffle partitions become 50 MB total → merged to 25 partitions. (2) Dynamic skew join handling — detects skewed partitions (>spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes) and splits them. Scenario: user_id='unknown' has 50% of data → split into sub-partitions automatically. (3) Dynamic join strategy switching — converts SortMergeJoin to BroadcastHashJoin when the build side turns out small after filtering. Scenario: filtered table drops from 10 GB to 50 MB → auto-broadcast.

■ Key Code / Config:

```
spark.conf.set('spark.sql.adaptive.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.enabled', 'true')
spark.conf.set('spark.sql.adaptive.coalescePartitions.enabled', 'true')
spark.conf.set('spark.sql.adaptive.skewJoin.skewedPartitionFactor', '5')
spark.conf.set('spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes', '256MB')
```

■ *AQE requires at least one shuffle stage to collect runtime statistics. It cannot optimize plans with no shuffle.*

Section 2: Memory Management & Spill

Q9 Describe Spark's Unified Memory Model. How does it allocate memory between execution and storage, and what happens during memory pressure?

■ *Asked at: Uber · Lyft · Grab* | ■ *Difficulty: Hard*

■ Answer:

Spark's Unified Memory Model (introduced in 1.6) divides JVM heap into: (1) Reserved Memory (~300 MB) — for Spark internals. (2) User Memory (1 - spark.memory.fraction) * heap — for user data structures, not managed by Spark. (3) Spark Memory (spark.memory.fraction, default 0.6) — split between Execution (shuffles, sorts, joins) and Storage (cached data). The key innovation is the unified boundary — execution can borrow from storage memory and vice versa. Under memory pressure: execution memory can evict cached blocks from storage. However, execution memory cannot be evicted once in use (it would cause incorrect results). Storage memory can only borrow execution memory when execution pool is idle. Result: if both are under pressure, storage evicts to disk, execution spills to disk.

■ Key Code / Config:

```
# Memory fractions
spark.conf.set('spark.memory.fraction', '0.6')       # Spark-managed pool
spark.conf.set('spark.memory.storageFraction', '0.5') # Initial storage within Spark poo
l
# Total executor memory budget:
# spark.executor.memory + spark.executor.memoryOverhead (for off-heap, Python, etc.)
```

■ *For shuffle-heavy workloads, lower spark.memory.storageFraction to give execution more initial memory.*

Q10 You see 'spill to disk' warnings in your Spark job. Explain the different types of spill and the performance impact of each.

■ *Asked at: Meta · Amazon · Pinterest* | ■ *Difficulty: Hard*

■ Answer:

Two types of spill: (1) Shuffle spill — occurs during sort/hash operations when the in-memory buffer is full. Data is sorted and written to disk in multiple sorted runs, then merged on read. Measured in Spark UI as 'Shuffle Write Spill' and 'Shuffle Read Spill'. (2) Storage spill — when cached DataFrames with MEMORY_AND_DISK storage level evict partitions to disk due to memory pressure. Performance impact: disk I/O is 10-100x slower than memory. If spill is significant, total job time can increase 3-5x. Remediation: (a) Increase executor memory. (b) Reduce partition size by increasing shuffle partition count. (c) Enable off-heap memory to reduce GC interference. (d) For aggregations, use reduceByKey instead of groupByKey (pre-aggregates before shuffle). (e) Consider bucketing to eliminate shuffle altogether.

■ Key Code / Config:

```
# Reduce spill by tuning partition size
spark.conf.set('spark.sql.shuffle.partitions', '4000')

# Monitor spill in Spark UI or programmatically
# Look for 'spill (memory)' and 'spill (disk)' columns in Stages tab
```

■ *A small amount of spill is acceptable. Alarm when spill (disk) > 10% of total shuffle write bytes.*

Q11 When would you use off-heap memory in Spark? What are the risks and trade-offs?

■ *Asked at: Cloudera · Databricks · IBM* | ■ *Difficulty: Hard*

■ Answer:

Off-heap memory lives outside the JVM heap — allocated via sun.misc.Unsafe and managed by Spark/Tungsten directly. When to use: (1) Very large executor memory (>32 GB) where GC pauses become significant. (2) Caching massive DataFrames that cause frequent GC. (3) High-frequency columnar operations benefiting from CPU cache locality. Trade-offs: Benefits: eliminates GC overhead for managed data, more predictable performance, can exceed JVM heap limits. Risks: (a) Memory leaks are harder to detect — no GC collection. (b) Native OOM errors bypass JVM error handling and can crash executors. (c) Debugging is harder — standard Java profilers don't see off-heap. (d) Interaction with Python UDFs still requires JVM serialization. Configuration: spark.memory.offHeap.enabled + spark.memory.offHeap.size.

■ Key Code / Config:

```
spark.conf.set('spark.memory.offHeap.enabled', 'true')
spark.conf.set('spark.memory.offHeap.size', '8g')  # Per executor

# Also increase memoryOverhead for container managers
# spark.executor.memoryOverhead = max(384MB, 0.1 * executor.memory)
```

■ *When using Kubernetes or YARN, always account for off-heap in container memory limits to avoid container kills.*

Q12 Your Spark executors are being killed by YARN/Kubernetes with 'Container killed on request' or OOMKilled. Walk through the diagnosis and fix.

■ *Asked at: Netflix · Spotify · Twilio* | ■ *Difficulty: Medium*

■ Answer:

Container kills happen when process memory exceeds container allocation. Two scenarios: (1) JVM heap OOM: task too large for executor heap → increase spark.executor.memory. (2) Off-heap OOM: Python processes, native libraries, shuffle buffers exceeding overhead → increase spark.executor.memoryOverhead (YARN) or spark.kubernetes.memoryOverheadFactor (K8s). Diagnosis steps: (a) Check container logs for OOMKilled signal. (b) Check Spark UI Executor tab for GC time and peak memory usage. (c) If Python UDFs used, Python process memory is not tracked by JVM. (d) Check for large broadcasts — spark.broadcast.blockSize and total broadcast size. Fix: Systematically increase overhead first (cheaper), then executor memory. For persistent OOMs, reduce spark.executor.cores to reduce concurrent task memory pressure.

■ Key Code / Config:

```
# YARN
spark.conf.set('spark.executor.memoryOverhead', '2g')

# Kubernetes
spark.conf.set('spark.kubernetes.memoryOverheadFactor', '0.2')  # 20% of executor memory

# Reduce concurrent task pressure
spark.conf.set('spark.executor.cores', '4')  # Not 8+ per executor
```

■ *Total container memory = spark.executor.memory + spark.executor.memoryOverhead. Request this as container size.*

Q13 Explain the difference between spark.executor.memory, spark.executor.memoryOverhead, and spark.memory.offHeap.size. How do they interact?

■ *Asked at: Twitter · Snap · TikTok* | ■ *Difficulty: Hard*

■ Answer:

Three separate memory pools: (1) spark.executor.memory — JVM heap size for the executor process. This is managed by the JVM GC and contains: Spark's unified memory (execution + storage) + user memory. (2) spark.executor.memoryOverhead — extra memory requested from the cluster manager (YARN/K8s) on top of executor memory. Used for: JVM overhead (metaspace, code cache), Python/R worker processes, native libraries (Arrow, TensorFlow), shuffle sort buffers outside heap. Default: max(384MB, 0.1 * executor.memory). (3) spark.memory.offHeap.size — a portion of off-heap memory used by Tungsten for managed off-heap allocation. Must be enabled with spark.memory.offHeap.enabled=true. This comes out of memoryOverhead budget, so increase overhead when using off-heap. Interaction: Container memory = executor.memory + memoryOverhead. Off-heap competes within memoryOverhead budget.

■ Key Code / Config:

```
# Complete memory config example
spark.conf.set('spark.executor.memory', '16g')      # JVM heap
spark.conf.set('spark.executor.memoryOverhead', '4g') # Native + Python
spark.conf.set('spark.memory.offHeap.enabled', 'true')
spark.conf.set('spark.memory.offHeap.size', '2g')    # Within overhead budget
# Container allocation: 16 + 4 = 20 GB
```

■ *If you enable off-heap (2g), increase memoryOverhead by at least that amount to avoid container OOM.*

Q14 A Spark job fails with 'java.lang.OutOfMemoryError: GC overhead limit exceeded'. How is this different from a regular heap OOM and how do you fix it?

■ *Asked at: Databricks · Palantir · Cloudera* | ■ *Difficulty: Medium*

■ Answer:

GC overhead limit exceeded is a JVM heuristic: when >98% of CPU time is spent in GC and <2% of heap is recovered, the JVM throws this error rather than continuing in an ineffective GC loop. This often precedes full heap OOM. Key difference: regular OOM means no memory available; GC overhead means memory is technically available but fragmented and unreclaimable. Root causes: (1) Too many live small objects (RDD of Row objects). (2) Long-lived object references preventing GC (accumulators, broadcast variables). (3) String-heavy datasets without interning. Fixes: (a) Switch to DataFrame API (Tungsten avoids per-Row objects). (b) Enable G1GC: -XX:+UseG1GC -XX:InitiatingHeapOccupancyPercent=35. (c) Increase executor memory. (d) Repartition to reduce objects per task. (e) Disable GC overhead limit for debugging: -XX:-UseGCOverheadLimit (not production fix).

■ Key Code / Config:

```
# JVM GC tuning in spark-defaults.conf
# spark.executor.extraJavaOptions=
#   -XX:+UseG1GC
#   -XX:G1HeapRegionSize=32m
#   -XX:+PrintGCDetails
#   -XX:InitiatingHeapOccupancyPercent=35

# Quick fix: increase memory and use DataFrame API
spark.conf.set('spark.executor.memory', '32g')
```

■ *Enable GC logging to diagnose: -XX:+PrintGCDateStamps -Xloggc:/tmp/gc.log, then analyze with GCViewer.*

Section 3: Shuffles, Joins & Skew Handling

Q15 You have a 500 GB fact table joining a 5 TB dimension table on user_id. 0.1% of user_ids account for 60% of rows (extreme skew). How do you handle this?

■ *Asked at: Uber · DoorDash · Meta* | ■ *Difficulty: Hard*

■ Answer:

This is a severe skew problem. Standard SortMergeJoin will have a few tasks running for hours. Solutions: (1) Salting technique: add a random salt key (0 to N-1) to the skewed keys in the large table, then replicate the small table N times with all salt values. This distributes skewed partitions across N tasks. (2) AQE skew join (automatic): enable spark.sql.adaptive.skewJoin.enabled — Spark auto-detects and splits skewed partitions, no code change needed. (3) Pre-filter skewed keys: process the top-K skewed user_ids separately (broadcast their lookup), process rest normally, then union results. (4) Two-phase aggregation: pre-aggregate at partition level before the join to reduce data volume.

■ Key Code / Config:

```
# Salting approach
N = 50  # salt factor
fact_salted = fact.withColumn('salt', (F.rand() * N).cast('int')) \
                  .withColumn('skew_key', F.concat('user_id', F.lit('_'), 'salt'))

dim_replicated = dim.withColumn('salt', F.explode(F.array([F.lit(i) for i in range(N)]))
) \
                    .withColumn('skew_key', F.concat('user_id', F.lit('_'), 'salt'))

result = fact_salted.join(dim_replicated, 'skew_key')
```

■ *With AQE enabled, try the automatic approach first — it avoids manual salting complexity.*

Q16 Explain the difference between BroadcastHashJoin, SortMergeJoin, and ShuffledHashJoin. When does Spark choose each?

■ *Asked at: LinkedIn · Airbnb · Zalando* | ■ *Difficulty: Hard*

■ Answer:

Three join strategies in Spark: (1) BroadcastHashJoin (BHJ): One table is broadcast to all executors — no shuffle needed. Chosen when: one side < spark.sql.autoBroadcastJoinThreshold (default 10 MB). Fastest for eligible cases. (2) SortMergeJoin (SMJ): Both tables sorted on join key, then merged. Chosen when: both sides are large and join keys are sortable. Requires two shuffle stages. Most common for large-large joins. (3) ShuffledHashJoin (SHJ): Shuffle both sides, build hash map from smaller side, probe with larger. Chosen when: one side is small enough to build hash map in memory but too large to broadcast. Between SMJ and BHJ in performance. Spark's cost-based optimizer selects based on estimated statistics. You can hint the planner: df.hint('broadcast'), df.hint('merge'), df.hint('shuffle_hash').

■ Key Code / Config:

```
# Force broadcast
result = large_df.join(F.broadcast(small_df), 'id')

# Increase broadcast threshold
spark.conf.set('spark.sql.autoBroadcastJoinThreshold', str(50 * 1024 * 1024))  # 50 MB

# Check chosen strategy
large_df.join(small_df, 'id').explain()
```

■ *BHJ is 5-10x faster than SMJ for eligible sizes. Always verify with explain() that broadcast is actually applied.*

Q17 Describe how bucketing works in Spark and when it eliminates shuffles. What are the limitations?

■ *Asked at: Google · Apple · Salesforce* | ■ *Difficulty: Hard*

■ Answer:

Bucketing pre-partitions data into N buckets based on a hash of a column, stored as files. When two bucketed tables have the same number of buckets and bucket column, a join between them requires no shuffle — each bucket from table A only joins with the corresponding bucket from table B. How to create: df.write.bucketBy(N, 'join_col').sortBy('join_col').saveAsTable('name'). Requirements for shuffle elimination: (1) Both tables must have same bucket count. (2) Join must be on bucket column(s). (3) Must be read as Hive tables (not as DataFrames from files). Limitations: (a) Doesn't work with CSV/JSON — only ORC/Parquet with metastore. (b) Bucket count must be chosen upfront and is hard to change. (c) Files must be managed by Spark catalog. (d) Small bucket count → large files; too many → many small files.

■ Key Code / Config:

```
# Write bucketed table
df.write.bucketBy(256, 'user_id') \
        .sortBy('user_id') \
        .saveAsTable('user_events_bucketed')

# Join will skip shuffle if both bucketed the same way
df1 = spark.table('user_events_bucketed')
df2 = spark.table('user_profiles_bucketed')
df1.join(df2, 'user_id').explain()  # Should show no Exchange
```

■ *Choose bucket count as a power of 2 >= expected parallelism. Common: 256, 512, 1024.*

Q18 What is a shuffle and why is it expensive? Walk through what happens at the byte level during a shuffle in Spark.

■ *Asked at: Netflix · Hulu · Disney+* | ■ *Difficulty: Hard*

■ Answer:

A shuffle is a full data redistribution across partitions — required by operations like groupBy, join, repartition, distinct. Step-by-step: (1) Map phase: each task writes its output records to a local shuffle file, partitioned by hash(key) % numReducePartitions. This creates N output files per mapper (N = num reduce partitions). (2) Shuffle service: Spark's external shuffle service (or executor) makes these files available via HTTP. (3) Reduce phase: each reducer task fetches its partition from all mapper outputs, deserializes, and processes. Expense factors: (a) Serialization/deserialization of every record. (b) Network I/O to fetch data from all executors. (c) Disk I/O to write/read shuffle files. (d) Hash computation for partitioning. (e) Number of shuffle files = mappers × reducers (can be millions).

■ Key Code / Config:

```
# Minimize shuffle with pre-aggregation
# Instead of:
df.groupBy('key').agg(F.collect_list('val'))  # All data shuffled

# Use partial aggregation where possible:
df.rdd.mapPartitions(partial_agg) \  # Pre-aggregate per partition
      .reduceByKey(merge_func)        # Final merge with less data
```

■ *Use spark.shuffle.compress=true and spark.shuffle.spill.compress=true to reduce shuffle file sizes.*

Q19 How would you join a 10 TB table with a 100 MB lookup table as efficiently as possible in Spark?

■ *Asked at: Databricks · Fivetran · dbt Labs* | ■ *Difficulty: Medium*

■ Answer:

With a 100 MB lookup table, BroadcastHashJoin is ideal but the default threshold (10 MB) is too low. Options: (1) Increase broadcast threshold: spark.sql.autoBroadcastJoinThreshold=200MB — Spark will auto-broadcast. (2) Explicit broadcast hint: df_large.join(F.broadcast(df_small), 'key') — overrides threshold, reliable. (3) If lookup table is a static reference dataset, load it as a broadcast variable and use a UDF or map-side join. (4) Pre-filter the lookup table before broadcasting to ensure it stays small. Trade-offs: Broadcasting sends data to ALL executors — with 200 executors this means 100MB × 200 = 20 GB of network traffic. For very wide clusters, a well-partitioned SMJ might actually be faster. Validate with explain() and timing.

■ Key Code / Config:

```
# Option 1: Explicit broadcast (recommended)
result = large_df.join(F.broadcast(lookup_df), 'product_id')

# Option 2: Threshold adjustment
spark.conf.set('spark.sql.autoBroadcastJoinThreshold', str(200 * 1024 * 1024))

# Verify it's broadcast
result.explain()  # Should show BroadcastHashJoin
```

■ *After filtering, a table that was 10 GB can drop to 50 MB — AQE can detect this at runtime and switch to broadcast automatically.*

Q20 You need to perform a self-join on a 2 TB table to find duplicate records. How do you do this efficiently?

■ *Asked at: Airbnb · Booking · Expedia* | ■ *Difficulty: Hard*

■ Answer:

A naive self-join on 2 TB creates a 4 TB shuffle problem. Efficient approach: (1) First, identify candidate duplicate keys using aggregation: compute a fingerprint (hash of all columns) and group by it — this is far cheaper than a full self-join. (2) If records are truly identical: use dropDuplicates() or window functions with row_number() partitioned by dedup key. (3) If fuzzy matching needed: use MinHash LSH for approximate similarity — reduces candidate pairs dramatically. (4) Repartition by the join key before self-join to co-locate matching records. (5) Use filter early to reduce data volume. (6) If identifying near-duplicates: sort-based merge is better than hash join for large datasets.

■ Key Code / Config:

```
from pyspark.sql import Window

# Efficient dedup using window
w = Window.partitionBy('record_hash').orderBy('created_at')
df_deduped = df.withColumn('rn', F.row_number().over(w)) \
               .filter('rn = 1') \
               .drop('rn')

# Or using built-in
df.dropDuplicates(['col1', 'col2', 'col3'])
```

■ *dropDuplicates() is more efficient than a self-join for exact dedup — it uses partial aggregation before shuffle.*

Q21 Explain data skew detection. How do you identify skew programmatically and what metrics indicate it?

■ *Asked at: Meta · Twitter · Reddit* | ■ *Difficulty: Hard*

■ Answer:

Skew indicators: (1) Spark UI: Stage detail shows tasks with wildly different durations — one task running 10x longer than median. (2) Shuffle read size distribution: most tasks read ~100 MB but one reads 50 GB. (3) GC time spike in specific tasks. Programmatic detection: (a) Check value distribution: df.groupBy('key').count().orderBy('count', ascending=False).show(). (b) Compute statistics: mean, stddev, percentiles — if P99/P50 > 10, skew is likely. (c) Use df.rdd.mapPartitions(lambda it: [sum(1 for _ in it)]).collect() to see partition size distribution. Fix decision: if <5% of keys cause skew → salting or separate processing. If widespread → rethink partitioning strategy.

■ Key Code / Config:

```
# Detect skew in a column
skew_analysis = df.groupBy('user_id') \
                  .count() \
                  .select(
                    F.mean('count').alias('mean'),
                    F.stddev('count').alias('stddev'),
                    F.max('count').alias('max'),
                    F.percentile_approx('count', [0.5, 0.9, 0.99]).alias('percentiles')
                  )
skew_analysis.show()
```

■ *Skew ratio = max_partition_size / avg_partition_size. If > 5, investigate. If > 10, must fix.*

Q22 How does the Spark shuffle service work in YARN and Kubernetes? What problems does it solve?

■ *Asked at: Spotify · Pandora · SoundCloud* | ■ *Difficulty: Hard*

■ Answer:

Problem it solves: Without external shuffle service, shuffle files are stored on executor JVM memory. When an executor is killed (due to dynamic allocation, failure, preemption), its shuffle data is lost, requiring recomputation. External Shuffle Service (ESS): A long-running service on each worker node (separate from executors) that stores and serves shuffle files. Benefits: (1) Enables dynamic allocation — executors can be released after map phase since shuffle files persist in ESS. (2) Faster recovery — reducer tasks can still fetch data even if mapper executor died. Kubernetes: ESS support added in Spark 3.x via spark.dynamicAllocation.shuffleTracking.enabled=true (doesn't require external service, tracks shuffle usage). Remote Shuffle Services (RSS): For large clusters, services like Celeborn (Apache Uniffle), Cosco (Uber) provide centralized shuffle storage with better fault tolerance.

■ Key Code / Config:

```
# YARN: enable external shuffle service
spark.conf.set('spark.shuffle.service.enabled', 'true')
spark.conf.set('spark.dynamicAllocation.enabled', 'true')

# Kubernetes: shuffle tracking (no separate service needed)
spark.conf.set('spark.dynamicAllocation.shuffleTracking.enabled', 'true')
spark.conf.set('spark.dynamicAllocation.enabled', 'true')
```

■ *For cloud environments (EMR, Dataproc), external shuffle service is typically pre-configured. Verify it's running before enabling dynamic allocation.*

Section 4: Streaming & Fault Tolerance

Q23 Explain the exactly-once semantics guarantee in Structured Streaming. How is it achieved end-to-end?

■ *Asked at: Uber · LinkedIn · Confluent* | ■ *Difficulty: Hard*

■ Answer:

Structured Streaming achieves exactly-once through three mechanisms: (1) Idempotent sources: Sources must be replayable (Kafka with offset tracking, file-based sources). Spark records consumed offsets in the checkpoint. On restart, it replays from the last committed offset. (2) Write-ahead log (WAL) / checkpoint: After each micro-batch, Spark atomically writes the batch metadata (offsets processed, batch ID) to a checkpoint directory before writing output. (3) Idempotent sinks: Output writes must be idempotent or transactional. Kafka sink uses transaction API. Delta Lake uses transaction log. File sinks use atomic rename. Together: If a failure occurs mid-batch, Spark replays the batch from source (idempotent) and re-writes to sink (idempotent/transactional) — result is identical to successful first write.

■ Key Code / Config:

```
query = df.writeStream \
    .format('kafka') \
    .option('kafka.bootstrap.servers', 'broker:9092') \
    .option('topic', 'output') \
    .option('checkpointLocation', 'hdfs://checkpoints/stream1') \
    .outputMode('append') \
    .start()
# Checkpoint stores: offsets, metadata, committed batches
```

■ *Exactly-once is NOT guaranteed if sink is not idempotent. Writing to a non-transactional DB without upsert logic gives at-least-once.*

Q24

Your Structured Streaming job has a growing checkpoint directory and increasing batch latency over time. What could cause this and how do you fix it?

■ *Asked at: Netflix · DoorDash · Grab* | ■ *Difficulty: Hard*

■ Answer:

Root causes of growing checkpoint and latency: (1) Checkpoint compaction not running: WAL files accumulate — enable spark.sql.streaming.minBatchesToRetain and ensure compaction triggers. (2) State store growth: Stateful operations (aggregations, stream-stream joins) accumulate state indefinitely — set watermark to expire old state. (3) Kafka lag: Upstream produces faster than Spark consumes — tune maxOffsetsPerTrigger. (4) Trigger interval too short: Processing overhead exceeds trigger interval causing queuing. (5) State backend performance: default in-memory state store for large state → use RocksDB state store. Fixes: (a) Set eventTime watermark to bound state. (b) Switch to RocksDB: spark.sql.streaming.stateStore.providerClass=RocksDBStateStoreProvider. (c) Increase trigger interval. (d) Clean old checkpoint files periodically.

■ Key Code / Config:

```
# Bounded state with watermark
df.withWatermark('event_time', '10 minutes') \
  .groupBy(F.window('event_time', '5 minutes'), 'user_id') \
  .count()

# RocksDB state store
spark.conf.set(
  'spark.sql.streaming.stateStore.providerClass',
  'org.apache.spark.sql.execution.streaming.state.RocksDBStateStoreProvider'
)
```

■ *Monitor streaming metrics: processingTime, batchDuration, numInputRows, stateRows via StreamingQueryListener.*

Q25 Describe the watermark mechanism in Structured Streaming. What are its limitations and edge cases?

■ *Asked at: Airbnb · Twitter · Lyft* | ■ *Difficulty: Hard*

■ Answer:

Watermark is a threshold that tells Spark how late data can arrive. For a watermark of W minutes, Spark maintains state for events up to W minutes behind the maximum observed event time. It serves two purposes: (1) State cleanup — events older than watermark are dropped from state. (2) Aggregate finalization — window results are emitted only after the watermark passes the window end. Limitations and edge cases: (a) Global watermark: uses the minimum watermark across all partitions — a single slow partition prevents state cleanup. (b) Late data is silently dropped — no error or alerting by default. (c) In Update mode, intermediate results are emitted before window closes — may need Complete mode for accurate final counts. (d) Stream-stream join watermark: both sides need watermarks, and the join state grows until the outer watermark advances. (e) Time zones: watermark uses UTC internally — ensure event_time is in consistent timezone.

■ Key Code / Config:

```
# Watermark with window aggregation
df.withWatermark('event_time', '2 hours') \
  .groupBy(
    F.window('event_time', '1 hour', '30 minutes'),  # window, slide
    'product_id'
  ).agg(F.sum('amount').alias('total'))
# Results emitted when watermark passes window_end
```

■ *The watermark delay trades completeness for latency. Larger watermark = more complete results but higher state memory.*

Q26 How do you implement a stream-stream join in Structured Streaming? What are the state management requirements?

■ *Asked at: Meta · Snap · TikTok* | ■ *Difficulty: Hard*

■ Answer:

Stream-stream joins require buffering both streams because neither side is fully known at any point. Requirements: (1) Both streams must have watermarks defined to bound state growth. (2) A time constraint on the join condition is mandatory for state to be cleaned. (3) Output mode must be append. State management: Spark buffers records from both streams waiting for matches. State for each record is kept until: (a) A match is found and emitted. (b) The watermark advances past the time when a match can no longer arrive (based on time constraint). Without time constraint: state grows unboundedly and OOM will eventually occur. Supported join types: inner (matching records), left/right outer (with watermark bounds). Not supported: full outer without watermark, cross joins.

■ Key Code / Config:

```
# Stream-stream join with state bounds
orders = spark.readStream.format('kafka')...
payments = spark.readStream.format('kafka')...

orders_wm = orders.withWatermark('order_time', '1 hour')
payments_wm = payments.withWatermark('payment_time', '30 minutes')

joined = orders_wm.join(
    payments_wm,
    expr("""
      order_id = payment_order_id AND
      payment_time BETWEEN order_time AND order_time + INTERVAL 1 HOUR
    """)
)
```

■ *Monitor stateOperatorProgress in StreamingQueryProgress for state row count — runaway state is an early warning.*

Q27

A Structured Streaming job reading from Kafka suddenly starts processing very slowly after a weekend (Kafka has 72 hours of backlog). How do you handle this safely?

■ *Asked at: Confluent · Databricks · AWS* | ■ *Difficulty: Hard*

■ Answer:

This is the 'backlog recovery' problem. Naively processing 72 hours of backlog causes: (a) Huge micro-batches overwhelming executors. (b) State store explosions if stateful. (c) Downstream systems overwhelmed. Safe handling: (1) Rate limiting: set maxOffsetsPerTrigger to limit records per batch — gradually increase as backlog clears. (2) Trigger.Once or Trigger.AvailableNow: process one batch at a time with manual progression — allows monitoring and rate control. (3) Separate catch-up job: run a batch job to process the backlog first, then restart streaming from the recovered offset. (4) Increase parallelism temporarily: more executors during catch-up, then scale down. (5) Monitor consumer group lag in Kafka to track progress.

■ Key Code / Config:

```
# Rate limit during catch-up
query = df.writeStream \
    .trigger(processingTime='30 seconds') \
    .option('maxOffsetsPerTrigger', 100000) \
    .option('checkpointLocation', 'hdfs://chk/') \
    .start()

# Or process available data in batches
query = df.writeStream \
    .trigger(availableNow=True) \
    .start()
query.awaitTermination()
```

■ *Trigger.AvailableNow (Spark 3.3+) processes all available data in multiple batches then stops — ideal for scheduled incremental catch-up.*

Q28 Explain Spark's lineage graph (DAG). How does it enable fault tolerance and what are its limitations?

■ *Asked at: Uber · Grab · Gojek* | ■ *Difficulty: Hard*

■ Answer:

Spark's DAG (Directed Acyclic Graph) records the logical derivation of each partition from its parents. Fault tolerance mechanism: If a partition is lost (executor failure), Spark recomputes it by traversing the lineage back to a materialized checkpoint and re-executing only the necessary transformations. Two types of dependencies: (1) Narrow dependency: each child partition depends on one parent partition (map, filter) — fast recovery, parallel recomputation. (2) Wide dependency: child partitions depend on multiple parent partitions (shuffle) — expensive recovery, may require recomputing many parent partitions. Limitations: (a) Long lineage chains → slow recovery (recompute from scratch). (b) Iterative algorithms create deep lineages → checkpoint periodically. (c) If input source data is lost (HDFS block lost), lineage-based recovery fails. (d) Wide dependency failures require recomputing entire upstream stage.

■ Key Code / Config:

```
# Checkpoint to truncate lineage (iterative algorithms)
sc.setCheckpointDir('hdfs://checkpoints/')

for i in range(100):
    rdd = rdd.map(update_step)
    if i % 10 == 0:
        rdd = rdd.checkpoint()  # Materialize, truncate lineage
        rdd.count()  # Force checkpoint to complete
```

■ *Use persist() + checkpoint() together: persist() keeps data in memory for current use, checkpoint() saves to disk to truncate lineage.*

Q29 What is speculative execution in Spark? When should you enable it and when should you NOT?

■ *Asked at: Netflix · Amazon · Microsoft* | ■ *Difficulty: Medium*

■ Answer:

Speculative execution detects straggler tasks (running significantly longer than other tasks in the same stage) and launches duplicate copies on different executors. The first copy to finish wins; others are killed. When to enable: (1) Jobs with occasional hardware failures causing slow disks/CPUs on some nodes. (2) Cloud environments with preemptible/spot instances. (3) Large jobs where one straggler blocks entire pipeline. Configuration: spark.speculation=true, spark.speculation.multiplier (default 1.5 — task is 1.5x slower than median), spark.speculation.quantile (default 0.75 — 75% of tasks must finish before speculating). When NOT to enable: (a) Non-idempotent operations (writing to external DBs) — could double-write. (b) Skewed data — stragglers are expected, speculation wastes resources. (c) Limited resources — duplicate tasks compete with progress.

■ Key Code / Config:

```
spark.conf.set('spark.speculation', 'true')
spark.conf.set('spark.speculation.multiplier', '2.0')  # 2x slower than median
spark.conf.set('spark.speculation.quantile', '0.90')   # 90% tasks done before speculati
ng
spark.conf.set('spark.speculation.interval', '100ms')
```

■ *Check if stragglers are due to skew before enabling speculation — if so, fix skew instead.*

Q30 How do you implement idempotent writes in a Spark streaming pipeline writing to a relational database?

■ *Asked at: Lyft · Airbnb · Stripe* | ■ *Difficulty: Hard*

■ Answer:

Idempotent writes ensure that re-processing a batch (on failure) produces the same result, not duplicate records. Strategies: (1) Upsert (MERGE/INSERT OR REPLACE): use batch_id + record_id as primary key. On retry, upsert replaces existing records. (2) Batch ID tracking: store processed batch IDs in a separate table. Before writing, check if batch_id already exists — if so, skip. (3) Truncate-and-reload: for small result sets, truncate the target partition and reload on every batch (idempotent by definition). (4) Deduplication in sink: if using Delta Lake/Iceberg, use MERGE INTO for upsert semantics. (5) Transactional writes: use database transactions — begin, write, commit. On retry, transaction not committed → safe to retry. foreachBatch is the recommended API for custom sink logic in Structured Streaming.

■ Key Code / Config:

```
def write_to_db(batch_df, batch_id):
    # Idempotent upsert using batch_id
    batch_df_with_id = batch_df.withColumn('batch_id', F.lit(batch_id))

    batch_df_with_id.write \
        .format('jdbc') \
        .option('url', JDBC_URL) \
        .option('dbtable', 'temp_batch') \
        .mode('overwrite') \
        .save()

    # Merge in DB (atomic)
    execute_sql(f'MERGE INTO target USING temp_batch ON ...')

query = df.writeStream.foreachBatch(write_to_db).start()
```

■ *Always include batch_id in your write logic. It is automatically available in foreachBatch and enables safe replay.*

Section 5: Delta Lake, Lakehouse & Storage

Q31 Explain Delta Lake's transaction log. How does it provide ACID guarantees on top of object storage?

■ *Asked at: Databricks · Netflix · Adobe* | ■ *Difficulty: Hard*

■ Answer:

Delta Lake uses a JSON/Parquet-based transaction log (_delta_log/) stored alongside data files to provide ACID properties. Structure: Each commit creates a new log entry (000000.json, 000001.json...) recording: added/removed files, schema changes, operation metadata. ACID guarantees: (1) Atomicity: writes either fully commit (log entry written) or are invisible (no log entry). Partial writes result in uncommitted data files that readers ignore. (2) Consistency: schema validation enforced at commit time. (3) Isolation: Optimistic concurrency control — conflicts detected at commit via file-level conflict resolution. Two transactions can write to non-overlapping partitions concurrently. (4) Durability: transaction log on object storage (S3/ADLS/GCS) is inherently durable. Snapshot isolation: readers see the table as of a specific log version, regardless of concurrent writers.

■ Key Code / Config:

```
# View transaction history
spark.sql('DESCRIBE HISTORY delta.`/path/to/table`').show()

# Time travel
df = spark.read.format('delta') \
           .option('versionAsOf', 10) \
           .load('/path/to/table')

# Or by timestamp
df = spark.read.format('delta') \
           .option('timestampAsOf', '2024-01-15') \
           .load('/path/to/table')
```

■ *Checkpoint files (.checkpoint.parquet) are created every 10 commits to speed up log replay — avoid deleting them.*

Q32 You are implementing a MERGE operation on a 5 TB Delta table with 1 million small source records. The job takes 2 hours. How do you optimize it?

■ *Asked at: Uber · Lyft · Databricks* | ■ *Difficulty: Hard*

■ Answer:

Slow MERGE on large Delta tables is a common problem. Root causes: (1) Full table scan: without proper filtering, MERGE scans all 5 TB. (2) Too many small files in target: file listing and processing overhead. (3) Shuffle of entire table to match source records. Optimizations: (1) Partition pruning: add partition filter to MERGE condition — Spark will only scan relevant partitions. (2) Z-ordering + data skipping: if target is Z-ordered on join key, file skipping dramatically reduces scan. (3) Low Shuffle Merge (Delta 2.0+): spark.databricks.delta.merge.enableLowShuffle.merge=true — only shuffles changed data, not entire table. (4) Reduce file count with OPTIMIZE before MERGE to improve scan efficiency. (5) Use insertOnly mode if you only insert new records (no updates) — faster than full MERGE.

■ Key Code / Config:

```
-- Partition-aware MERGE
MERGE INTO target t
USING source s
ON t.user_id = s.user_id
  AND t.date >= '2024-01-01'  -- partition pruning!
WHEN MATCHED THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *;

-- Enable low shuffle merge (Databricks)
SET spark.databricks.delta.merge.enableLowShuffle.merge = true;
```

■ *Low Shuffle Merge can reduce MERGE time by 50-80% for workloads where <20% of target rows are touched.*

Q33 Describe the small files problem in data lakes. How does it manifest, what causes it, and how do you prevent and cure it?

■ *Asked at: Netflix · Meta · Spotify* | ■ *Difficulty: Hard*

■ Answer:

Small files problem: thousands/millions of tiny files (< 10 MB) in a storage layer cause massive metadata overhead — file listing, open/close operations dominate over actual data reads. Causes: (1) Streaming micro-batches writing many small files per batch. (2) Partitioning on high-cardinality columns (date + hour + minute + user_id). (3) Appending without compaction. (4) Overwriting individual records (non-MERGE upserts). Manifestation: slow table scans, high HDFS NameNode load, Hive metastore timeouts, S3 ListObjects rate limits hit. Prevention: (a) Coalesce output: df.coalesce(N).write.parquet(). (b) Correct partition granularity — never partition on columns with >10K distinct values. (c) Schedule regular OPTIMIZE jobs. (d) For streaming: use longer trigger intervals or batch compaction jobs. Cure: Delta OPTIMIZE, Apache Hudi compaction, Iceberg rewrite.

■ Key Code / Config:

```
-- Delta Lake: compact small files
OPTIMIZE delta.`/path/to/table`
  WHERE date >= current_date() - INTERVAL 7 DAYS;

# PySpark: coalesce before write
df.coalesce(100).write.mode('append').parquet('path/')

# Or repartition by output key
df.repartition(200, 'date').write.partitionBy('date').parquet('path/')
```

■ *Target file size: 128 MB–1 GB for batch. Use spark.sql.files.maxPartitionBytes=128MB to guide read parallelism.*

Q34 Compare Delta Lake, Apache Iceberg, and Apache Hudi. When would you choose each?

■ *Asked at: Databricks · Microsoft · AWS* | ■ *Difficulty: Hard*

■ Answer:

All three are open table formats providing ACID, schema evolution, and time travel on data lakes. Delta Lake: Strengths — deep Spark integration, excellent read/write performance, mature ecosystem (DBR, Unity Catalog), Liquid Clustering. Weaknesses — historically Databricks-centric (now fully open), less multi-engine support. Use when: Spark-first, Databricks environment, heavy DML (MERGE/UPDATE/DELETE). Apache Iceberg: Strengths — best multi-engine support (Spark, Flink, Trino, Dremio, BigQuery), metadata layer with no file listing bottleneck, hidden partitioning. Weaknesses — more complex operational model. Use when: multi-engine lakehouse, vendor-neutral, query engine diversity needed. Apache Hudi: Strengths — streaming upserts, record-level index for near-real-time updates, Copy-on-Write vs Merge-on-Read flexibility. Weaknesses — complex operational tuning, slower reads for MOR tables. Use when: CDC pipelines, frequent record-level upserts, real-time ingestion.

■ Key Code / Config:

```
# Iceberg: hidden partitioning (auto-maintains as data evolves)
spark.sql("""
  CREATE TABLE catalog.db.events (
    user_id BIGINT, event_time TIMESTAMP, ...
  ) USING ICEBERG
  PARTITIONED BY (days(event_time))  -- auto hidden partition
""")
```

■ *For new projects in 2025+: Iceberg is increasingly the default choice for vendor-neutral multi-engine architectures.*

Q35 How do you implement schema evolution safely in a Spark pipeline reading Delta/Iceberg tables?

■ *Asked at: Meta · Netflix · Airbnb* | ■ *Difficulty: Hard*

■ Answer:

Schema evolution risks: adding columns is safe; removing, renaming, or changing types can break downstream consumers. Safe practices: (1) Delta Lake: enable mergeSchema for additive changes — spark.write.option('mergeSchema', 'true'). For breaking changes, use overwriteSchema=true with full awareness. (2) Iceberg: schema evolution is metadata-only (no data rewrite) — supports add column, drop column, rename, reorder, promote type (int→long). (3) Contracts: use schema registries (Confluent, AWS Glue) to enforce compatibility checks before writes. (4) Backward compatibility rule: only add nullable columns to existing positions; never change types incompatibly. (5) Column mapping (Delta 2.0+): maps column names to IDs, allowing rename/drop without rewriting data. (6) Test pipelines against schema changes in staging before production deploy.

■ Key Code / Config:

```
# Delta: add column without rewrite
df_new.write.format('delta') \
    .option('mergeSchema', 'true') \
    .mode('append') \
    .save('/path/to/delta')

-- Iceberg: safe schema evolution
ALTER TABLE catalog.db.events
  ADD COLUMN new_field STRING AFTER existing_field;
```

■ *Use ALTER TABLE ADD COLUMN rather than schema overwrite — it's always backwards compatible and metadata-only.*

Q36 Your Spark pipeline writes to partitioned Parquet on S3. After 6 months, query performance degrades significantly. Diagnose and fix.

■ *Asked at: Databricks · AWS · Google* | ■ *Difficulty: Medium*

■ Answer:

Likely causes after 6 months of appends: (1) Small files accumulation: thousands of tiny files per partition from daily/hourly appends. (2) Stale statistics: Spark's cost-based optimizer uses outdated stats. (3) Partition over-granularity: too many small partitions. (4) No file format optimization: uncompressed or sub-optimal compression. Diagnosis: (a) Count files per partition: hadoop fs -count /path/partition=*. (b) Check average file size — anything < 64 MB is suspect. (c) Run ANALYZE TABLE to refresh statistics. Fix: (1) Compact: run periodic Spark job to read and rewrite with coalesce. (2) Migrate to Delta/Iceberg with OPTIMIZE. (3) Review partition strategy — if partitioned by hour, consider daily + Z-order on hour. (4) Switch to zstd compression for better ratio. (5) Schedule weekly maintenance job.

■ Key Code / Config:

```
# Compaction job
for partition_value in get_all_partitions():
    spark.read.parquet(f's3://bucket/table/date={partition_value}') \
         .coalesce(target_files) \
         .write.mode('overwrite') \
         .parquet(f's3://bucket/table/date={partition_value}')

# Update table stats
spark.sql('ANALYZE TABLE my_table COMPUTE STATISTICS FOR ALL COLUMNS')
```

■ *Migrate to Delta Lake — OPTIMIZE handles compaction efficiently and incrementally, avoiding full partition rewrites.*

Q37 Explain Change Data Capture (CDC) implementation patterns in Spark. How do you handle deletes?

■ *Asked at: Netflix · Hulu · Peacock* | ■ *Difficulty: Hard*

■ Answer:

CDC captures insert, update, delete operations from a source (typically database binlog via Debezium/DMS). Challenges: (1) Ordering: CDC events must be applied in correct order. (2) Deduplication: retransmissions and at-least-once delivery create duplicates. (3) Deletes: hard to handle in append-only lake. Patterns: (1) MERGE pattern (Delta/Iceberg/Hudi): read CDC batch, apply MERGE to target table — handles inserts, updates, and deletes atomically. Use op field (I/U/D) to determine action. (2) Soft deletes: add is_deleted flag + updated_at; filter in queries. Clean up with periodic MERGE. (3) Hudi MOR: designed for CDC — records merged at read time using record-level index. (4) Iceberg row-level deletes: DELETE FROM with predicate, no data rewrite. Best practice: always include a sequence/timestamp field for ordering and use MERGE WHEN MATCHED AND source.op='D' THEN DELETE.

■ Key Code / Config:

```
-- Delta CDC MERGE with delete handling
MERGE INTO target t
USING cdc_batch s ON t.id = s.id
WHEN MATCHED AND s.op = 'D' THEN DELETE
WHEN MATCHED AND s.op = 'U' THEN UPDATE SET *
WHEN NOT MATCHED AND s.op = 'I' THEN INSERT *;
```

■ *Use sequence_num or lsn from CDC source to deduplicate before MERGE — process only the latest operation per key.*

Section 6: Cluster Management & Scheduling

Q38 Explain dynamic resource allocation in Spark. What are its benefits and potential problems in production?

■ *Asked at: Uber · Airbnb · Booking* | ■ *Difficulty: Hard*

■ Answer:

Dynamic Resource Allocation (DRA) allows Spark to request and release executors based on workload. How it works: (1) Spark monitors the scheduler backlog — if tasks are pending for >spark.dynamicAllocation.schedulerBacklogTimeout, request more executors. (2) If executors are idle for >spark.dynamicAllocation.executorIdleTimeout, release them. Benefits: (a) Better cluster utilization — executors released when not needed. (b) Cost reduction in cloud environments. (c) Multiple jobs can share cluster resources. Production problems: (a) Shuffle data loss: released executors take their shuffle files. Requires external shuffle service or shuffle tracking. (b) Slow scale-up for bursty workloads: requesting new executors from YARN/K8s takes time. (c) Premature scale-down: idle detection may release executors between stages, causing re-requests. (d) Cache invalidation: cached data on released executors must be recomputed. Fix: tune timeout values and always enable external shuffle service.

■ Key Code / Config:

```
spark.conf.set('spark.dynamicAllocation.enabled', 'true')
spark.conf.set('spark.dynamicAllocation.minExecutors', '5')
spark.conf.set('spark.dynamicAllocation.maxExecutors', '100')
spark.conf.set('spark.dynamicAllocation.initialExecutors', '10')
spark.conf.set('spark.shuffle.service.enabled', 'true')  # Required!
```

■ *Set minExecutors > 0 for jobs with short idle periods to avoid slow scale-up overhead between stages.*

Q39 How do you configure Spark on Kubernetes for production workloads? What are the key differences from YARN?

■ *Asked at: LinkedIn · Twitter · Salesforce* | ■ *Difficulty: Hard*

■ Answer:

Key Kubernetes considerations: (1) Pod resource limits: each executor is a pod with CPU/memory requests and limits. Set limits = requests for predictable scheduling. (2) Image management: all code/dependencies must be in the Docker image — no distributed NFS needed. (3) Shuffle storage: no persistent shuffle service by default — use shuffle tracking (3.x) or external RSS (Celeborn). (4) Namespace and RBAC: Spark driver needs permissions to create/delete executor pods (spark-operator simplifies this). (5) Node affinity and tolerations: pin executors to specific node pools for predictable performance. (6) Persistent volumes for checkpoint storage (ReadWriteMany — NFS/EFS). Key differences from YARN: (a) No NameNode bottleneck. (b) Pod startup latency (10-30s) vs YARN container (1-5s). (c) Storage is decoupled — no HDFS; use object storage. (d) Fine-grained resource isolation per pod. (e) Better for multi-tenant, mixed-workload environments.

■ Key Code / Config:

```
# spark-submit for K8s
spark-submit \
  --master k8s://https://k8s-api:443 \
  --deploy-mode cluster \
  --conf spark.kubernetes.container.image=my-spark:3.5 \
  --conf spark.kubernetes.namespace=spark-jobs \
  --conf spark.executor.instances=20 \
  --conf spark.kubernetes.executor.request.cores=2 \
  --conf spark.kubernetes.executor.limit.cores=2 \
  local:///app/job.py
```

■ *Use spark-operator (kubeflow) for production — it handles CRDs, driver failure recovery, and log collection automatically.*

Q40 Explain the FAIR scheduler vs FIFO scheduler in Spark. How do you configure pools for multi-tenant environments?

■ *Asked at: Netflix · Databricks · Cloudera* | ■ *Difficulty: Hard*

■ Answer:

FIFO (default): Jobs are processed in submission order. First job gets all resources; subsequent jobs wait. Simple but unfair for mixed-priority workloads. FAIR scheduler: Multiple pools with configurable weights and minShare guarantees. Jobs within a pool scheduled round-robin. Benefits: (a) Low-priority jobs don't starve high-priority ones. (b) Minimum guarantees for critical pipelines. (c) Weight-based proportional allocation for burst capacity. Configuration: spark.scheduler.mode=FAIR, then define pools in fairscheduler.xml. Use cases: (1) Assign interactive queries high weight for fast response. (2) Give ETL jobs minimum share for baseline throughput. (3) Separate pools for different teams/SLAs. Limitation: FAIR applies within a single SparkContext — for true multi-tenancy, use separate applications with cluster-level queues (YARN capacity scheduler).

■ Key Code / Config:

```
# fairscheduler.xml
#
#   FAIR
#   3
#   10
#

# Submit to specific pool
spark.conf.set('spark.scheduler.pool', 'production')

# Or per-job
sc.setLocalProperty('spark.scheduler.pool', 'interactive')
```

■ *For Databricks, use cluster pools and job clusters instead of FAIR scheduler — more isolation and predictable performance.*

Q41 How do you size Spark executors? Discuss the fat executor vs thin executor trade-off.

■ *Asked at: Airbnb · Grab · Gojek* | ■ *Difficulty: Medium*

■ Answer:

Fat executor (many cores, lots of memory per executor): Pros: better data locality within executor (shared cache), less overhead per executor (fewer JVMs). Cons: Higher GC pressure, single executor failure loses more partitions, HDFS throughput limited (>5 cores per executor sees diminishing returns). Thin executor (few cores, less memory): Pros: more parallelism for small jobs, better fault isolation. Cons: More JVM overhead, worse data locality. Recommended sweet spot: 4-5 cores per executor, 16-32 GB memory, leaving 1 core + some memory per node for OS/HDFS daemons. Formula: if node has 64 GB RAM, 16 cores → use 3 executors per node × (5 cores, 19 GB each), leaving 1 core + 7 GB for OS. For memory-intensive workloads: fewer cores per executor (2-3) to reduce per-task memory competition. For CPU-bound: more cores (6-8) with sufficient memory.

■ Key Code / Config:

```
# Recommended config for 64GB / 16-core nodes
spark.conf.set('spark.executor.cores', '5')
spark.conf.set('spark.executor.memory', '18g')
spark.conf.set('spark.executor.memoryOverhead', '2g')
# 3 executors per node, 1 core for OS
```

■ *HDFS client throughput saturates at 5 concurrent tasks — no benefit to >5 cores/executor if HDFS-bound.*

Q42 What is the driver's role in Spark and what causes driver OOM? How do you prevent it?

■ *Asked at: Meta · Google · Microsoft* | ■ *Difficulty: Hard*

■ Answer:

The driver runs the SparkContext, schedules tasks, and collects results. It never processes data partition contents — it only manages metadata. Driver OOM causes: (1) .collect() or .toPandas() on large DataFrames — brings all data to driver heap. (2) Huge broadcast variables — broadcast data is first materialized on driver. (3) Large accumulators — each accumulator result is sent to driver. (4) Too many cached partitions metadata — Spark tracks partition locations on driver. (5) Collecting results of show()/display() on huge datasets. Prevention: (a) Never use collect() on large data — use write() instead. (b) Limit broadcast size: spark.sql.autoBroadcastJoinThreshold wisely. (c) Increase driver memory: spark.driver.memory=8g for complex jobs. (d) Use take(N) instead of collect(). (e) For iterative result inspection, write to storage and query.

■ Key Code / Config:

```
# WRONG - OOM risk
all_data = df.collect()  # Millions of rows to driver!

# RIGHT - write to storage
df.write.parquet('s3://output/')

# RIGHT - sample for inspection
df.sample(0.001).toPandas()  # Only 0.1% to driver

# Increase driver memory
spark.conf.set('spark.driver.memory', '16g')
```

■ *Set spark.driver.maxResultSize (default 1 GB) to fail fast if a collect() would OOM the driver.*

Q43 How do you monitor and debug a slow or hung Spark job in production? Walk through the systematic debugging process.

■ *Asked at: Databricks · Cloudera · IBM* | ■ *Difficulty: Hard*

■ Answer:

Systematic debugging process: (1) Spark UI first: check Jobs tab → identify which stage is slow/hung. Check Stage detail → task duration distribution, shuffle metrics, spill. Check Executor tab → GC time, memory usage, failed tasks. (2) Identify the bottleneck type: (a) Data skew: few tasks take 100x longer → fix skew. (b) Shuffle bottleneck: high shuffle read time → reduce shuffle size, tune partitions. (c) Spill: large spill (disk) values → increase memory. (d) GC: high GC time → tune GC, reduce object creation. (e) Network: high shuffle fetch wait → network saturation, tune locality. (3) Hung job: check for deadlocks — circular dependencies in accumulators or shared state. Check for Kafka consumer group rebalancing blocking streaming. (4) Tools: spark.eventLog.enabled=true for post-mortem. Ganglia/Prometheus for cluster-level metrics. Log4j DEBUG for verbose task logging.

■ Key Code / Config:

```
# Enable event log for debugging
spark.conf.set('spark.eventLog.enabled', 'true')
spark.conf.set('spark.eventLog.dir', 'hdfs:///spark-logs')

# Add custom metrics
from pyspark.accumulators import AccumulatorParam
counter = sc.accumulator(0)

# Check task metrics programmatically
df.queryExecution.executedPlan.metrics
```

■ *Spark History Server is invaluable for post-mortem analysis. Always enable event log in production.*

Section 7: Advanced APIs, SQL & Architecture

Q44 Explain the Catalyst optimizer pipeline. How does a SQL query become an executed Spark job?

■ *Asked at: Netflix · Meta · Uber* | ■ *Difficulty: Hard*

■ Answer:

Catalyst processes a query through 4 phases: (1) Analysis: parses SQL/DSL into an Unresolved Logical Plan → resolves column names and types against catalog → Resolved Logical Plan. (2) Logical Optimization: applies rule-based optimizations — predicate pushdown, column pruning, constant folding, boolean simplification, subquery elimination. Produces Optimized Logical Plan. (3) Physical Planning: generates multiple Physical Plans from the logical plan, applies cost-based optimizer (CBO) to select the best one — chooses join strategy, ordering, etc. (4) Code Generation: Whole-Stage Code Generation compiles the physical plan into Java bytecode using Janino. Multiple operators fused into a single tight loop. The compiled code is then executed by executor JVMs. You can inspect each phase: df.queryExecution.analyzed, .optimizedPlan, .executedPlan.

■ Key Code / Config:

```
# Inspect each Catalyst phase
plan = df.queryExecution
print('Analyzed:', plan.analyzed)
print('Optimized:', plan.optimizedPlan)
print('Physical:', plan.executedPlan)
print('Codegen:', plan.executedPlan.toJSON())

# Or use explain with mode
df.explain(mode='extended')  # Shows all phases
```

■ *Catalyst is extensible — you can add custom optimization rules via experimental.extraOptimizations for domain-specific rewrites.*

Q45 What are Spark's Dataset encoders? How do they differ from DataFrame rows and when should you use typed APIs?

■ *Asked at: Databricks · Google · Amazon* | ■ *Difficulty: Hard*

■ Answer:

Dataset uses Encoders to convert between JVM objects (case classes, POJOs) and Spark's internal binary row format. Encoders know the schema at compile time — generated by Encoders.product[T] or implicitly in Scala. How they differ from DataFrame rows: DataFrame uses Row objects (dynamic schema, no compile-time type safety). Dataset uses typed objects (compile-time safety, IDE support). Both use Tungsten's binary format internally — no JVM object overhead during computation. When to use typed APIs: (a) Complex domain logic with multiple transformations on specific types (better refactoring, compile errors vs runtime). (b) When you need map/filter with arbitrary Scala/Java functions that aren't expressible in SQL. (c) ML feature engineering pipelines with custom types. When NOT to use: Python (no Datasets in PySpark), performance-critical paths with complex types (encoder overhead), SQL-heavy analytics.

■ Key Code / Config:

```
# Scala Dataset example
case class UserEvent(userId: Long, eventType: String, amount: Double)

val ds: Dataset[UserEvent] = spark.read.json('events/').as[UserEvent]
val filtered: Dataset[UserEvent] = ds.filter(_.amount > 100.0)

# Custom encoder
implicit val enc = Encoders.product[UserEvent]
```

■ *In Python, use DataFrame API — PySpark has no Dataset. Python UDFs can be wrapped as Pandas UDFs for vectorized execution.*

Q46

You need to compute a complex session window for user activity: sessions end after 30 minutes of inactivity. How do you implement this efficiently in Spark?

■ *Asked at: Airbnb · Lyft · Pinterest* | ■ *Difficulty: Hard*

■ Answer:

Session window computation on streaming data is a classic problem. Options: (1) Spark 3.2+ Native Session Window: spark.sql.session.windowFunctions.enabled=true — uses session_window(timestamp, '30 minutes'). Handles state management and late data with watermark. (2) Batch approach with window functions: for each user, order events by timestamp, compute lag, mark session boundaries where gap > 30 min, use cumulative sum of boundary markers as session_id. (3) Custom stateful processing: flatMapGroupsWithState (Structured Streaming) — maintain per-user state of last event time and current session. Emit session record when 30-min timeout detected. Batch approach is simplest for non-streaming data:

■ Key Code / Config:

```
from pyspark.sql import Window

w = Window.partitionBy('user_id').orderBy('event_time')

df_sessions = df \
  .withColumn('prev_time', F.lag('event_time').over(w)) \
  .withColumn('gap_secs',
    (F.col('event_time').cast('long') - F.col('prev_time').cast('long'))) \
  .withColumn('new_session',
    F.when(F.col('gap_secs') > 1800, 1).otherwise(0)) \
  .withColumn('session_id',
    F.sum('new_session').over(w.rowsBetween(Window.unboundedPreceding, 0))) \
  .withColumn('session_key',
    F.concat('user_id', F.lit('_'), 'session_id'))
```

■ *For streaming session windows, use watermark slightly larger than session gap to handle late events.*

Q47 How do you implement an incremental processing pipeline in Spark without Structured Streaming? What patterns exist for batch incremental loads?

■ *Asked at: Twitter · Reddit · Quora* | ■ *Difficulty: Hard*

■ Answer:

Batch incremental patterns without streaming: (1) Watermark-based (high-water mark): store last_processed_timestamp in a metadata table. Each run, read WHERE updated_at > last_watermark. Update watermark after successful write. Works for CDC-captured data with reliable timestamps. (2) Partition-based incremental: process only new/changed date partitions. Use partition listing to find partitions newer than last run. (3) File-based: track processed file paths in a manifest. Only process new files (useful for S3 landing zones). (4) Delta CDF (Change Data Feed): use DESCRIBE HISTORY to find last processed version, use df.read.format('delta').option('readChangeFeed', 'true').option('startingVersion', N) to read only changes. (5) Spark autoLoader: cloud-native incremental file processing with checkpoint-based tracking.

■ Key Code / Config:

```
# Delta CDF incremental
last_version = get_last_processed_version()

changes = spark.read.format('delta') \
    .option('readChangeFeed', 'true') \
    .option('startingVersion', last_version + 1) \
    .table('source_table')

process_and_write(changes)
update_last_version(get_current_version())

# AutoLoader for file ingestion
df = spark.readStream \
    .format('cloudFiles') \
    .option('cloudFiles.format', 'parquet') \
    .load('s3://landing-zone/')
```

■ *Delta CDF is the most reliable incremental pattern — version numbers are monotonic and gaps are impossible.*

Q48 Explain Spark's support for semi-structured data (JSON, XML, nested arrays). What are the performance pitfalls?

■ *Asked at: Meta · LinkedIn · Salesforce* | ■ *Difficulty: Hard*

■ Answer:

Spark handles nested data through StructType, ArrayType, MapType in schemas. Common operations: explode() to flatten arrays, get_json_object() for dynamic JSON extraction, schema_of_json() for schema inference. Performance pitfalls: (1) Schema inference on JSON: spark.read.json() samples data to infer schema — expensive on large files. Always provide schema explicitly with .schema(). (2) explode() on large arrays: dramatically increases row count — do aggregations before explode when possible. (3) from_json() in streaming: parses JSON string per row — expensive if schema is complex. (4) Deeply nested structs: selecting nested fields creates struct copies in Tungsten — flatten early in the pipeline. (5) Schema inference with corrupt records: use PERMISSIVE mode + _corrupt_record column to handle malformed JSON without failing the job.

■ Key Code / Config:

```
from pyspark.sql.types import *

# Always provide schema - never infer in production
schema = StructType([
  StructField('id', LongType()),
  StructField('events', ArrayType(StructType([
    StructField('ts', TimestampType()),
    StructField('action', StringType())
  ])))
])

df = spark.read.schema(schema).json('data/')

# Flatten arrays efficiently
df.select('id', F.explode('events').alias('event')) \
  .select('id', 'event.ts', 'event.action')
```

■ *Use .select('nested.field') to extract struct fields — Spark pushes down the column selection to avoid materializing the full struct.*

Q49 How do you implement a medallion architecture (Bronze/Silver/Gold) in Spark + Delta Lake? What are the data quality patterns at each layer?

■ *Asked at: Databricks · Palantir · Snowflake* | ■ *Difficulty: Hard*

■ Answer:

Medallion architecture has three layers: Bronze (raw): exact replica of source data, no transformation. Append-only, never delete. Purpose: audit trail, reprocessing. Quality checks: file format validation, schema enforcement (rejects structurally invalid). Captured: ingestion timestamp, source file path, schema version. Silver (cleaned/conformed): deduplicated, type-cast, standardized, NULL-handled, business rules applied. Quality checks: null checks, range validation, referential integrity, duplicate detection. Reject invalid records to quarantine table. Gold (business-level aggregates): serving layer — aggregated metrics, wide tables for BI, feature tables for ML. Quality checks: metric consistency (totals match silver), SLA-based freshness checks, statistical anomaly detection (Z-score on daily metrics). Implementation: Use Delta MERGE for SCD Type 2 in Silver, partition Gold by query dimensions.

■ Key Code / Config:

```
# Silver layer: dedup + validate
def silver_transform(bronze_df):
    return bronze_df \
        .dropDuplicates(['id', 'event_time']) \
        .filter(F.col('amount') > 0) \
        .filter(F.col('user_id').isNotNull()) \
        .withColumn('processed_at', F.current_timestamp())

# Gold layer: aggregate
def gold_transform(silver_df):
    return silver_df \
        .groupBy('date', 'product_id') \
        .agg(F.sum('amount').alias('daily_revenue'),
             F.countDistinct('user_id').alias('dau'))
```

■ *Never transform Bronze — always derive Silver from Bronze. This allows reprocessing with new business rules without re-ingestion.*

Q50

Design a Spark pipeline for processing 100 TB/day of clickstream data with <1 hour latency SLA, guaranteed exactly-once, and support for late-arriving events up to 4 hours. What would your architecture look like?

■ *Asked at: All FAANG + Databricks* | ■ *Difficulty: Hard*

■ Answer:

Architecture overview: Ingest: Kafka with 200+ partitions (match Spark parallelism). Each event has event_time + kafka_offset for ordering. Processing: Structured Streaming with Trigger.ProcessingTime('5 minutes') for near-real-time. Watermark of 4 hours to handle late arrivals. RocksDB state store for large state. Idempotent writes via batch_id to Delta Lake. Exactly-once: Kafka offsets in checkpoint + Delta transaction log ensures no duplicates on failure. 4-hour watermark ensures late events are captured. Storage: Bronze layer: raw Kafka → Delta (append, keyed by kafka offset). Silver layer: dedup, watermark-bounded aggregation → Delta MERGE with batch_id. Gold layer: pre-aggregated metrics for BI. Infrastructure: Kubernetes with 100+ executors, auto-scaling. Remote shuffle service (Celeborn) for shuffle fault tolerance. Checkpoint on high-throughput HDFS. Monitoring: Streaming query listener with Prometheus metrics, PagerDuty alerts on consumer lag > 30 min.

■ Key Code / Config:

```
query = (
  spark.readStream
    .format('kafka')
    .option('kafka.bootstrap.servers', BROKERS)
    .option('subscribe', 'clickstream')
    .option('maxOffsetsPerTrigger', 10_000_000)
    .load()
    .select(F.from_json('value', schema).alias('e'), 'offset', 'partition')
    .select('e.*', 'offset', 'partition')
    .withWatermark('event_time', '4 hours')
    .groupBy(F.window('event_time', '1 hour'), 'user_id')
    .agg(F.count('*').alias('clicks'))
    .writeStream
    .foreachBatch(upsert_to_delta)
    .trigger(processingTime='5 minutes')
    .option('checkpointLocation', CHECKPOINT)
    .start()
)
```

■ *At 100 TB/day scale, partition count and shuffle service reliability are the top concerns. Budget for 3x peak load capacity.*

■ Quick Reference: Spark Config Cheat Sheet

Config Key	Recommended Value	Purpose
`spark.sql.adaptive.enabled`	true	Enable AQE
`spark.sql.adaptive.skewJoin.enabled`	true	Auto skew handling
`spark.sql.shuffle.partitions`	1000-4000	Scale with data size
`spark.executor.memory`	16-32g	Per executor heap
`spark.executor.memoryOverhead`	2-4g	Native + Python
`spark.memory.offHeap.enabled`	true (large clusters)	Reduce GC
`spark.serializer`	KryoSerializer	Faster serialization
`spark.sql.autoBroadcastJoinThreshold`	50MB	Broadcast threshold
`spark.speculation`	true (cloud)	Handle stragglers
`spark.dynamicAllocation.enabled`	true	Auto-scale executors
`spark.shuffle.service.enabled`	true	Required for DRA
`spark.sql.files.maxPartitionBytes`	128MB	Partition target size

Good luck with your interviews! Remember: the best answers combine theoretical knowledge with practical experience. Always mention trade-offs, real-world constraints, and monitoring strategies.